MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Multi-Agent Systems & Self-Reflection

Teams of agents, agents that critique themselves — and the evidence for when either is worth it.

Unit 08 · Session 12 · Week 12

Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **One agent or many?** the honest default, and three real reasons to split
2. **Architectures** supervisor-workers, handoffs, debate; frameworks conceptually
3. **Anti-patterns & evidence** sprawl, telephone games, cost; when teams win or lose
4. **Self-reflection** Reflexion, the limits of self-critique, external signals

Where we are

Unit 08 — from one observable agent to teams and self-critics

Last week (S11) — we made one agent observable: traces, spans, cost accounting, replay

Already seen — S06 introduced worker orchestration (orchestrator-workers, fan-out/fan-in) as a workflow pattern

Today — coordination and failure modes of agent teams, and agents critiquing their own work

Staying honest — both ideas are heavily oversold; we will follow evidence, not demos

GUIDING QUESTION

When does a second model call buy you something a better single agent cannot?

1

One agent or many?

The honest default — and the three reasons that actually justify a team

The honest default: one agent

Default architecture — one loop, one context, all tools — it covers most production use cases

Why it wins — the full history sits in one context window; nothing is lost in transfer

Industry guidance — start with the simplest design; add agents only when one clearly plateaus (Anthropic, 2024)

When it strains — context overflow, conflicting roles in one prompt, work that could run in parallel

Burden of proof — multi-agent is the option that must justify itself — never the starting point

Three reasons that can justify a team

Possible motives, not exclusive abilities: a single loop can also route models (S05) or call tools in parallel (S03)

Context isolation

Each agent gets a clean window. Long tool outputs, large documents, and noisy traces stay local instead of crowding one shared context.

Competence and authority

A role is defined by task, context and authority: what it sees, which tools it may use. Model choice is an additional design variable, not the definition.

Independent subtasks

Subtasks with no data dependency can run as separate agents at once. Pays only when they are genuinely independent and the merge is cheap.

2

Architectures

Three coordination patterns, and what the frameworks add conceptually

Three coordination patterns

Who decides what happens next?

Supervisor-workers

One orchestrator decomposes the task, spawns workers, integrates results. Single point of control — and of failure. The dominant production pattern.

Peer handoffs

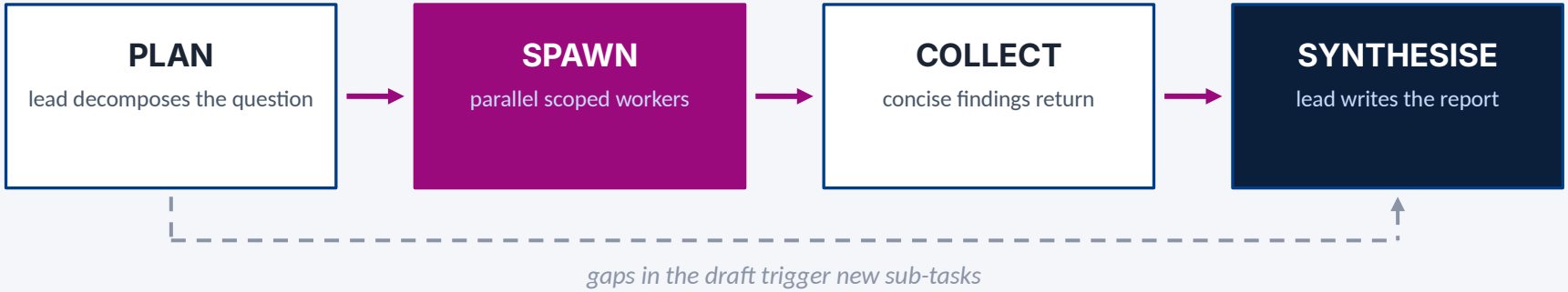
No boss: each agent finishes its part and passes control plus state to the next, like triage to specialist. Control travels with the conversation.

Debate and critique

Agents argue positions or review each other's drafts. Measurable factuality gains on some reasoning tasks (Du et al., 2023) — at multiplied cost.

Example: supervisor-workers for research

One way to lay out a breadth-first web-research task over many leads (cf. Anthropic, 2025)



- Each worker owns a clean context and a narrow brief: one subtopic, named tools, an output format.
- Workers return concise findings with source IDs and evidence excerpts; the underlying artefacts stay accessible.
- A design choice, not a law: the lead mostly manages briefs and budgets, but may sample sources to resolve conflicts.

Roles in a team: planner, workers, reviewer

A role is defined by task, context and authority; model choice is an additional design variable

Planner — decomposes the task, scopes effort per subtask, owns the budget

Workers — search and read one subtask each; return findings with source IDs and excerpts

Reviewer — receives task, draft and evidence; flags unsupported claims, gaps, contradictions

Why distance helps — withholding the drafter's reasoning may reduce anchoring; it does not guarantee independence

Correlated errors — same model, same sources: separate contexts do not create statistical independence

Model per role — often one model in different contexts; routing a cheaper or stronger model per role is an option (S05)

DESIGN RULE

Define a role by what it sees, what it may touch, and what it must output. A reviewer without evidence cannot verify grounding.

Frameworks: concepts over APIs

What each one believes coordination should look like

Framework	Core abstraction	Coordination model	Honest assessment
CrewAI	role-based crew: role, goal, backstory	sequential or manager-led process	fast start; opinionated, hides its prompts
AutoGen (now Microsoft Agent Framework)	conversable agents exchanging messages	group chat with a manager agent	conversations drift; AutoGen in maintenance since Oct 2025, successor MAF 1.0
LangGraph	explicit state graph, typed shared state	you wire nodes and edges yourself	most control and code; least hidden behaviour
Plain code	loops and function calls	whatever you write	fine for supervisor-workers; you own everything

Recall S07: the coordination concepts transfer between frameworks; the APIs do not.

3

Anti-patterns & evidence

How teams go wrong, and what the data says about when they help

Anti-patterns: how teams go wrong

Each one is common enough to have a name

Agent sprawl

Roles multiply because adding an agent feels like progress: a 'summariser', a 'formatter', a 'QA agent'. Each is a prompt that could have been a function — and each adds latency, cost, and a new failure surface.

Telephone game

Information degrades at every handoff: the worker's hedged nuance becomes the summary's claim becomes the report's fact. Prose handoffs are lossy; structured artefacts and shared stores reduce the loss but do not remove it.

Cost explosion

N agents means roughly N contexts re-reading overlapping material. Anthropic's internal measurement of its own research system (June 2025): about 15x the tokens of a chat.

Shared-state races

Parallel agents writing one document or memory store interleave edits nondeterministically. Two workers 'fix' the same paragraph differently; nobody notices until the report contradicts itself.

The price of a team

One production multi-agent system: roughly 15x the tokens of a chat.

Anthropic's internal measurement of its own research system (blog, June 2025; baseline: a chat interaction): single agents about 4x chat usage, their multi-agent system about 15x. One system's numbers, not a constant of multi-agent systems. The task value must cover whatever premium you measure.

Where teams win, where they lose

The pattern behind the published evidence

Multi-agent helps

- breadth-first research: many independent leads
- parallel reading beats one long sequence
- Anthropic internal eval, June 2025: +90.2% relative vs single-agent
- that is a relative gain on one eval, not 90 points of accuracy
- debate lifts factuality on some tasks (Du et al., 2023)

Multi-agent hurts

- tightly coupled work — coding in one codebase
- every dependency becomes a lossy handoff
- errors compound across agents
- coordination overhead exceeds task value
- fair comparison: same base model, same token and time budget

Why multi-agent systems fail

MAST: a failure taxonomy from annotated traces (Cemri et al., 2025)

The study — annotated execution traces across popular frameworks; 14 recurring failure modes

Specification failures — vague task and role definitions; agents disobey or duplicate work

Inter-agent misalignment — agents withhold, ignore, or misread each other's contributions

Verification failures — weak or missing final checks let wrong results through

Headline finding — most annotated failures fall into these design categories; model limits still contribute

READ IT AS

An empirical taxonomy of annotated failed runs, not proof that model capability is irrelevant. Fix specifications and checks first; keep measuring model effects.

Minimal viable team: agent plus critic

Two contexts, one model; every returned draft carries the verdict that was given on exactly that draft

critic_loop.py

```
def run_with_critic(task, sources, max_rounds=2):
    """max_rounds = revision budget; each draft is reviewed"""
    if not isinstance(max_rounds, int) or max_rounds < 0:
        raise ValueError("max_rounds must be an int >= 0")
    draft_id, draft = 0, worker.run(task, sources)
    while True:
        # fresh context: task, draft, sources; no worker trace
        review = critic.run(task, draft, sources)
        check_schema(review)     # verdict pass|revise, issues
        review["draft_id"] = draft_id  # verdict bound to draft
        if review["verdict"] == "pass":
            return draft, review, "pass"
        if draft_id == max_rounds:        # budget spent, open
            return draft, review, "unresolved"
        draft = worker.run(task, sources, review["issues"])
        draft_id += 1
```

Reading the critic loop

The details that make a two-agent team trustworthy

Budget semantics — max_rounds counts revisions; critic calls are max_rounds + 1, so the bill is still bounded

Verdict bound to draft — draft_id travels with the review; a stale verdict can never be attached to a newer draft

Last draft — either re-review it (shown) or return early and mark it 'revised but not reviewed'; never claim a pass

Validate inputs — max_rounds is checked; check_schema rejects a review without a verdict or issues

Evidence for the critic — the critic receives the sources: without them it cannot verify grounding, only style

Fresh context — withholding the worker's reasoning reduces anchoring; same model and sources still correlate errors

VERDICTS

A verdict must be checkable: a list of unsupported claims and missing sections, not a quality score.

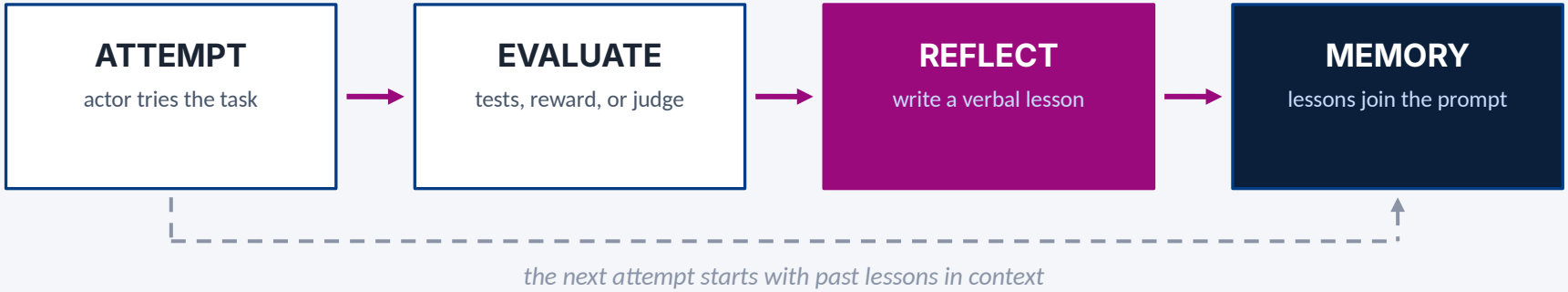
4

Self-reflection

Can an agent learn from its own failures — and whom should it ask?

The Reflexion loop (Shinn et al., 2023)

'Verbal reinforcement learning': inference-time adaptation through stored verbal feedback, no weight updates



- Feedback becomes language, not gradients: 'I searched too broadly; next time constrain the query.'
- An episodic memory buffer holds a few reflections, prepended to the next episode.
- No weights change: the lessons live in an episodic buffer that can be persisted and re-injected into later runs (cf. S08).

What Reflexion buys — and what it needs

Three modules — an actor, an evaluator, and a self-reflection model that writes the lesson

Reported gains — 91% pass@1 on HumanEval vs 80% GPT-4 baseline; near-perfect ALFWorld (Shinn et al., 2023)

Read with the protocol — actor plus evaluator, several trials with feedback, then the final test; not one unguided model call

The catch — gains hinge on a reliable evaluator: unit tests, environment success signals

Memory is small — a sliding window of recent lessons, not a database (cf. S08)

Why it matters — adaptation at inference time, without fine-tuning or gradient access

READ IT AS

Trial and error where the policy update is written down in English and re-read on the next attempt; the weights stay fixed.

Models grading their own homework

Intrinsic self-correction — asking a model to fix its own reasoning often reduces accuracy (Huang et al., 2024)

Why — the second pass comes from the same model with the same blind spots; its errors are correlated with the first

Shared blind spots — a fact the model believes wrongly tends to survive repeated self-review

Oracle leakage — several early self-correction results quietly used ground-truth labels to stop

What still works — style, format, completeness: surface properties the model can verify

RULE

Self-correction may fail because errors and blind spots are correlated. External evidence often helps; it is not optional for facts.

Reflection needs outside ground truth

External feedback signals, ordered by reliability

Executable checks

Unit tests, compilers, type checkers. Checkable signals within their coverage and trust assumptions: tests can be incomplete, wrong or edited by the agent. Keep them outside what the agent can change.

Retrieval and grounding

Verify claims against retrieved sources (CRITIC, Gou et al., 2024). The corpus arbitrates facts the model would happily self-confirm.

Environment outcomes

HTTP 200 or 'file written' proves the call ran, not that the task succeeded. Check the outcome itself: the right file content, the correctly booked transaction.

Humans

Slow and expensive, but the only signal for taste, intent, and stakes. Reserve human review for what no automated check can decide.

Lab 12: add a critic, measure it

Local Ollama (qwen2.5:7b), tiny offline corpus of invented facts, gap-fill notebook, 90 to 120 min

Part B baseline — a single-agent writer drafts a sourced Markdown report per topic; each call logs tokens

Part C critic — same model, fresh context, sees draft plus numbered sources; strict-JSON pass/revise verdict, MAX_ROUNDS = 2

Part D metric — grounding proxy: unsupported-claim rate (keyword overlap with the sources)

Part E measure — baseline vs critic on three topics: grounding, tokens, latency with S11-style cost accounting

Part F stretch — two parallel workers plus a merge; find the telephone game in the merged report

REPORT TASKS

R1 one agent or many; R2 half-page memo: did the critic pay for itself? Numbers, not impressions; R3 telephone game; R4 the critic prompt.

TAKEAWAYS

What you should remember

- **Default —** one agent, one context — multi-agent must earn its place
- **Three reasons —** context isolation, differing competence or authority, independent subtasks
- **Architectures —** supervisor-workers dominates; handoffs and debate are niche
- **Anti-patterns —** sprawl, telephone-game loss, state races; ~15x tokens in one vendor's system
- **Reflexion —** stored verbal feedback in episodic memory: inference-time adaptation, no weight updates
- **External beats internal —** tests, retrieval and outcomes are checkable within their coverage; self-opinion is not

NEXT SESSION Agent security — prompt injection and defences.

Reading & references

- Shinn, N. et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.
  The self-reflection paper: verbal feedback as the learning signal, episodic memory of lessons.
- Du, Y. et al. (2023). Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325.
  The debate result: multiple agents arguing measurably reduce factual errors.
- Huang, J. et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR 2024.
  The cold shower: intrinsic self-correction often hurts; earlier wins leaned on oracle labels.
- Gou, Z. et al. (2024). CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. ICLR 2024.
  Self-correction works when critique is grounded in external tools — search, interpreters.
- Anthropic (2025). How We Built Our Multi-Agent Research System. Engineering blog, June 2025.
  anthropic.com/engineering/multi-agent-research-system. Internal eval of one system: +90.2% relative vs single-agent, ~15x chat tokens.
- Cemri, M. et al. (2025). Why Do Multi-Agent LLM Systems Fail? NeurIPS 2025 Datasets & Benchmarks; arXiv:2503.13657.
  MAST failure taxonomy: specification, inter-agent misalignment, verification.
- Kim, Y. et al. (2025). Towards a Science of Scaling Agent Systems. arXiv:2512.08296.
  Controlled study: centralised coordination +81% on parallelisable, -39 to -70% on sequential tasks.
- Anthropic (2024). Building Effective Agents. anthropic.com/engineering. The start-simple guidance of section 1.